RAG-1: Retrieval-Augmented Generation — From First Principles

Complete Beginner-Friendly Notes

Topics covered: Why LLMs need RAG, Document Parsing, Chunking Strategies, Embeddings & Vector DBs, Naive RAG, Hybrid Search (BM25 + Semantic), Contextual Retrieval, Reranking, Query Transformation, Full Pipeline

---

The Big Picture — What is RAG and Why Does It Exist?

Imagine you have a super-smart friend who graduated college 2 years ago. They know a LOT — history, science, coding, math. But they don't know what happened in the news yesterday, they've never read your company's internal handbook, and when they don't know something, they sometimes confidently make stuff up instead of saying "I don't know."

That's exactly what an LLM is. RAG solves this by giving the LLM a cheat sheet right before it answers — you search your documents, find the relevant parts, and paste them into the prompt.

RAG in one sentence: Before asking the LLM a question, first search your documents for relevant info and paste it into the prompt so the LLM has the facts it needs.

The Core Problem: Three Reasons LLMs Need RAG

1. Frozen in Time: LLMs only know what was in their training data. They can't tell you about events after their training cutoff, recent policy changes, or today's stock price.

2. No Private Knowledge: The LLM has never seen your company handbook, your Slack messages, your internal documents, or your personal notes. That data doesn't exist in its world.

3. Confident Hallucination: When an LLM doesn't know something, instead of admitting ignorance, it often invents a plausible-sounding answer with full confidence. This is dangerous for anything factual.

RAG fixes all three: you retrieve the relevant facts from your own data and give them to the LLM, so it's answering from real documents rather than from memory (or imagination).

The RAG Workflow

The core flow is simple:

```
User Question → RETRIEVE relevant docs → AUGMENT the prompt with those docs → GENERATE an
swer
```

Here's the full cycle with numbered steps:

1. User asks a question
2. Your app converts the question into a search query
3. Search your document database (vector DB) for relevant chunks
4. Retrieved chunks are stuffed into the prompt alongside the question
5. The LLM reads the question + the retrieved context
6. The LLM generates an answer based on the provided context

When to Use RAG vs Alternatives

Strategy	When to Use
Context Window	Small knowledge base (under ~500 pages). Just paste it all into the prompt.
Fine-tuning	The model needs a new skill, writing style, or specific behavior pattern.
RAG	The model doesn't KNOW something — external data, private docs, recent info.

Fine-tuning teaches the model HOW to behave. RAG gives the model WHAT to know. They solve different problems and can be combined.

Ingestion Part 1: Document Loading & Parsing

The golden rule: Garbage In, Garbage Out.

Before you can search your documents, you need to convert them into clean text. This sounds simple but it's surprisingly hard:

- **PDFs:** Tables often get destroyed, columns merge, headers and footers repeat on every page
- **Word docs:** Structural info (headings, sections) gets lost, metadata disappears
- **HTML:** Navigation bars, footers, ads, cookie banners get mixed in with the actual content

If your parser mangles the text, everything downstream (chunking, embedding, retrieval) is built on a broken foundation.

Recommended parsing tools:

Tool	Best For
Unstructured.io	General purpose, handles many formats
LlamaParse	Complex PDFs with tables and charts
Docling	High-speed, structure-aware parsing

The notebook uses 5 sample documents from a fictional company (ACME Corp), pre-written as clean text strings. In a real system, you'd be loading and parsing actual files.

Ingestion Part 2: Chunking — Where 80% of RAG Systems Fail

You can't feed an entire 100-page document into the prompt (it wouldn't fit, and it would be wasteful). So you split documents into smaller pieces called "chunks." How you split them matters enormously.

Strategy 1: Fixed-Size Chunking (The Naive Way)

```
def chunk_fixed(text, chunk_size=200, overlap=50):
    words = text.split()
```

```
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i + chunk_size])
        chunks.append(chunk)
    return chunks
```

This splits text every N words with some overlap. The problem: it doesn't care about sentence or paragraph boundaries. A chunk might cut a sentence in half mid-thought: "The company reported revenue of $4.2 billion, which represents a" — and the next chunk continues "12% increase year-over-year."

Now neither chunk makes complete sense on its own. **Faithfulness score: ~0.47** (less than half the time, the retrieved chunk has the full answer).

Strategy 2: Recursive Chunking (The Smart Way)

```
def chunk_recursive(text, max_size=200):
    paragraphs = text.split("\n\n")  # Split on paragraph bre
aks first
    chunks = []
    for para in paragraphs:
        if len(para.split()) <= max_size:
            chunks.append(para)       # Paragraph fits → keep
it whole
        else:
            # Too long → split on sentences
            sentences = para.split(". ")
            # Group sentences to stay under max_size
            ...
    return chunks
```

This respects natural boundaries: first tries to split on paragraphs, then sentences, then words as a last resort. A chunk is a coherent thought, not an arbitrary cut.

Faithfulness score: ~0.80 — nearly double the naive approach.

Advanced Chunking Methods

- **Structure-aware:** Uses headings/sections from the document to create meaningful chunks
- **AST-based:** For code, splits on function/class boundaries rather than line counts
- **Parent-Child:** Embed small chunks (for precise matching) but retrieve their parent chunk (for full context)

Embeddings and Vector Storage

What are Embeddings?

An embedding converts text into a list of numbers (a vector) that captures its **meaning**. Similar text produces similar vectors.

For example:

- "I love dogs" → [0.23, -0.15, 0.87, ...]
- "I adore puppies" → [0.21, -0.14, 0.85, ...] (very close!)
- "The stock market crashed" → [-0.54, 0.72, -0.31, ...] (very different)

The notebook uses OpenAI's `text-embedding-3-small` model:

```
def get_embedding(text):
    resp = oai.embeddings.create(input=[text], model="text-em
bedding-3-small")
    return resp.data[0].embedding
```

What is a Vector Database?

A vector DB stores millions of these embeddings and lets you find the most similar ones in milliseconds. When a user asks a question, you embed the question, then search for chunks whose embeddings are closest in vector space.

The notebook uses **ChromaDB** (great for prototyping):

```
import chromadb
chroma = chromadb.Client()
```

```
collection = chroma.create_collection("naive_rag", metadata=
{"hnsw:space": "cosine"})

# Store chunks with their embeddings
collection.add(
    ids=["chunk_0", "chunk_1", ...],
    embeddings=embs,
    documents=all_chunks,
    metadatas=chunk_meta
)
```

The ecosystem:

Stage	Tools
Prototyping	ChromaDB, FAISS
Production	Pinecone, Weaviate, Qdrant
SQL-native	pgvector (PostgreSQL extension)

Always store metadata (source file, page number, date) alongside your vectors. You'll need it for filtering and citations.

Exercise 2: Naive RAG — Build It, Then Break It

The simplest RAG pipeline: chunk → embed → store → search → generate.

```
def naive_rag(question, k=5):
    # 1. Embed the question
    # 2. Find top-k similar chunks in ChromaDB
    results = collection.query(
        query_embeddings=[get_embedding(question)],
        n_results=k
    )

    # 3. Stuff chunks into a prompt
    context = "\n".join(results["documents"][0])
    prompt = f"Based on this context:\n{context}\n\nAnswer:
```

```
{question}"

    # 4. Send to LLM
    response = oai.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return response.choices[0].message.content
```

What Works

Simple factual questions from a single document work fine:

- "What was ACME's total revenue in Q3 2024?" → Finds the right chunk, gives correct answer.
- "What is the home office stipend?" → Correct.

What Breaks

Dates: "What changed on 2024-09-15?" — Semantic search doesn't understand dates as searchable strings. The embedding of "2024-09-15" doesn't look like the embedding of the paragraph that mentions that date.

Exact strings: "What is the Slack command to trigger a rollback?" — The command `/deploy rollback` is a specific string that semantic search can't match well.

Acronyms: "What was the RS256 migration?" — Embeddings don't handle technical acronyms or codes reliably.

Cross-document questions: "What is ACME's overall AI strategy?" — The answer is spread across multiple documents. Naive search might only find chunks from one.

Exercise 3: Hybrid Search — Semantic + BM25

The Fix for Exact Matches

BM25 is a classical keyword search algorithm (pre-AI era). It finds exact word matches. Semantic search finds meaning. Together, they cover each other's weaknesses.

Search Type	Good At	Bad At
Semantic (Dense)	Meaning, synonyms, concepts ("car" matches "automobile")	Exact dates, IDs, commands, acronyms
BM25 (Sparse)	Exact terms, names, IDs, dates ("2024-09-15" matches exactly)	Synonyms, rephrased questions

How They're Combined: Reciprocal Rank Fusion (RRF)

You run both searches, get two ranked lists, and merge them:

```
def reciprocal_rank_fusion(semantic_results, keyword_results,
k=60):
    scores = {}
    for rank, (idx, _) in enumerate(semantic_results):
        scores[idx] = scores.get(idx, 0) + 1 / (k + rank + 1)
    for rank, (idx, _) in enumerate(keyword_results):
        scores[idx] = scores.get(idx, 0) + 1 / (k + rank + 1)
    return sorted(scores.items(), key=lambda x: x[1], reverse
=True)
```

If a chunk appears in BOTH lists, its RRF score is higher. A chunk that's semantically relevant AND contains the exact keyword is almost certainly the right answer.

Results

After adding BM25, the previously broken queries start working:

- "What changed on 2024-09-15?" → BM25 matches the date string directly
- "What is the Slack command?" → BM25 finds `/deploy rollback` as a literal match
- "RS256 migration?" → BM25 catches the exact acronym

Industry consensus: Hybrid search beats semantic-only search every time.

Exercise 4: Contextual Retrieval — Anthropic's Technique

The Problem: Lost Context

When you chunk a document, each chunk loses the context of where it came from. A chunk saying "The company's revenue grew by 3%..." is meaningless without knowing WHICH company, WHICH quarter, WHICH year.

The Solution: Prepend Context Before Embedding

Before embedding each chunk, use an LLM to write a 2-3 sentence context header that captures what the chunk is about and where it came from:

```
def contextualize_chunk(chunk, full_doc, title):
    resp = anthropic.messages.create(
        model="claude-sonnet-4-20250514",
        messages=[{"role": "user", "content": f"""
            Here's a full document titled "{title}":
            {full_doc}

            Here's one chunk from it:
            {chunk}

            Write a SHORT context that situates this chunk
            within the full document. Include key identifiers
            like company name, time period, section topic.
        """}]
    )
    return f"[{resp.content[0].text}] {chunk}"
```

Before: "The company's revenue grew by 3%..."

After: "[From ACME Corp Q3 2024 financial report, Revenue section. Previous quarter revenue was $3.75B.] The company's revenue grew by 3%..."

Now the embedding captures BOTH the content and its context. This costs more during ingestion (one LLM call per chunk) but dramatically improves retrieval quality.

Result: 67% fewer retrieval failures when combined with BM25 and reranking.

Exercise 5: Reranking — The Precision Upgrade

The Two-Pass System

Think of it like a hiring process:

First pass (Bi-Encoder): Fast screening. Look at resumes individually, pick the top 50 candidates. This is what embedding search does — it encodes the question and documents separately, then compares them by distance.

Second pass (Cross-Encoder): Detailed interviews. Look at each candidate WITH the job description side by side. This is reranking — it sees the question and document TOGETHER, understanding nuance and context.

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v
2")

def rerank(question, results, top_k=3):
    pairs = [(question, r["chunk"]) for r in results]
    scores = reranker.predict(pairs)
    # Sort by cross-encoder score, take top 3
    ...
```

The cross-encoder is much slower (can't scan millions of docs), which is why we use it only on the top 10-20 results from the fast first pass. But it's much more accurate at picking the truly relevant chunks.

Contextual Compression

Even after reranking, a retrieved chunk might be 90% noise and 10% gold. Compression uses an LLM to extract only the relevant sentences before building

the final prompt. This saves tokens and reduces distraction.

Query Transformation — Making Questions Better

Sometimes the user's question isn't ideal for search. Four techniques to fix this:

1. Rewriting: Clean up messy questions. "hey so like what about that thing with the security stuff" → "What are ACME's security policies and requirements?"

2. Multi-Query: Generate 3-5 different phrasings of the same question, search with all of them, deduplicate results. This catches chunks that match one phrasing but not another.

3. HyDE (Hypothetical Document Embedding): Generate a fake answer first, then embed THAT instead of the question. Why? Because the fake answer looks more like actual document text than a question does — so it matches better. "What is ACME's revenue?" → fake answer: "ACME reported total revenue of approximately $X billion..." → embed this, search for similar chunks.

4. Step-Back Prompting: For specific questions, first ask a broader background question. "What was the RS256 migration?" → first search "ACME authentication system changes" to get context, then answer the specific question.

The Full Pipeline — Everything Together

```
def full_rag(question):
    # 1. Search: contextual chunks + hybrid (semantic + BM25)
→ top 10
    results = ctx_hybrid_search(question, k=10)

    # 2. Rerank: cross-encoder picks the best 3
    top = rerank(question, results, top_k=3)

    # 3. Generate: stuff top 3 into prompt, ask LLM
    context = "\n".join(f"[Source: {r['meta']['title']}]\n{r
['chunk']}" for r in top)
    prompt = f"""Answer based ONLY on the provided context. C
ite sources.
```

```
    If the context doesn't contain the answer, say so.

    Context:
    {context}

    Question: {question}"""

    response = oai.chat.completions.create(model="gpt-4o-min
i", messages=[...])
    return response
```

Good RAG Prompt Rules

1. **Grounding:** "Answer ONLY from the provided context" — prevents hallucination
2. **Escape hatch:** "If you don't know, say so" — better than making things up
3. **Citations:** "Cite your sources with metadata" — lets users verify
4. **Structure:** Label each chunk clearly so the LLM knows what it's reading

The End-to-End Pipeline

```
INGESTION (one-time):
  Documents → Parse → Chunk (recursive) → Contextualize (LLM) → Embed → Store in Vector D
B

RETRIEVAL (per query):
  Question → (optionally transform) → Hybrid Search (semantic + BM25) → Rerank (cross-enc
oder) → Top 3 chunks

GENERATION (per query):
  Prompt (question + top chunks + instructions) → LLM → Answer with Citations
```

Exercise 6: Side-by-Side Comparison

The notebook tests all three approaches on the same hard question:

> "What is the Slack rollback command and what are the security requirements for remote workers?"

This question requires finding an exact string (`/deploy rollback`) AND a semantic concept (security requirements) from different documents.

Approach	Exact Match?	Semantic Match?	Quality
Naive RAG (semantic only)	Misses the command	Finds some security info	Partial
Hybrid Search (+ BM25)	Finds the command	Finds security info	Good
Full Pipeline (+ contextual + rerank)	Finds the command	Best security chunks	Excellent

Each upgrade level adds measurable improvement.

The Upgrade Path

```
Level 0: Context Window          (small docs only, just paste everything)
Level 1: Naive Semantic Search   ← we started here
Level 2: Better Chunking         (recursive, respects boundaries)
Level 3: Contextual Retrieval    (Anthropic's technique, prepend context)
Level 4: Hybrid Search           (semantic + BM25 + RRF fusion)
Level 5: Reranking               (cross-encoder, precision upgrade)
Level 6: Query Transformation    (rewriting, multi-query, HyDE)
Level 7: Agentic RAG             (agent decides when/what to retrieve)
```

Each level fixes specific failure modes from the previous one. Start at Level 1, observe what breaks, then upgrade where needed.

Master Summary Table

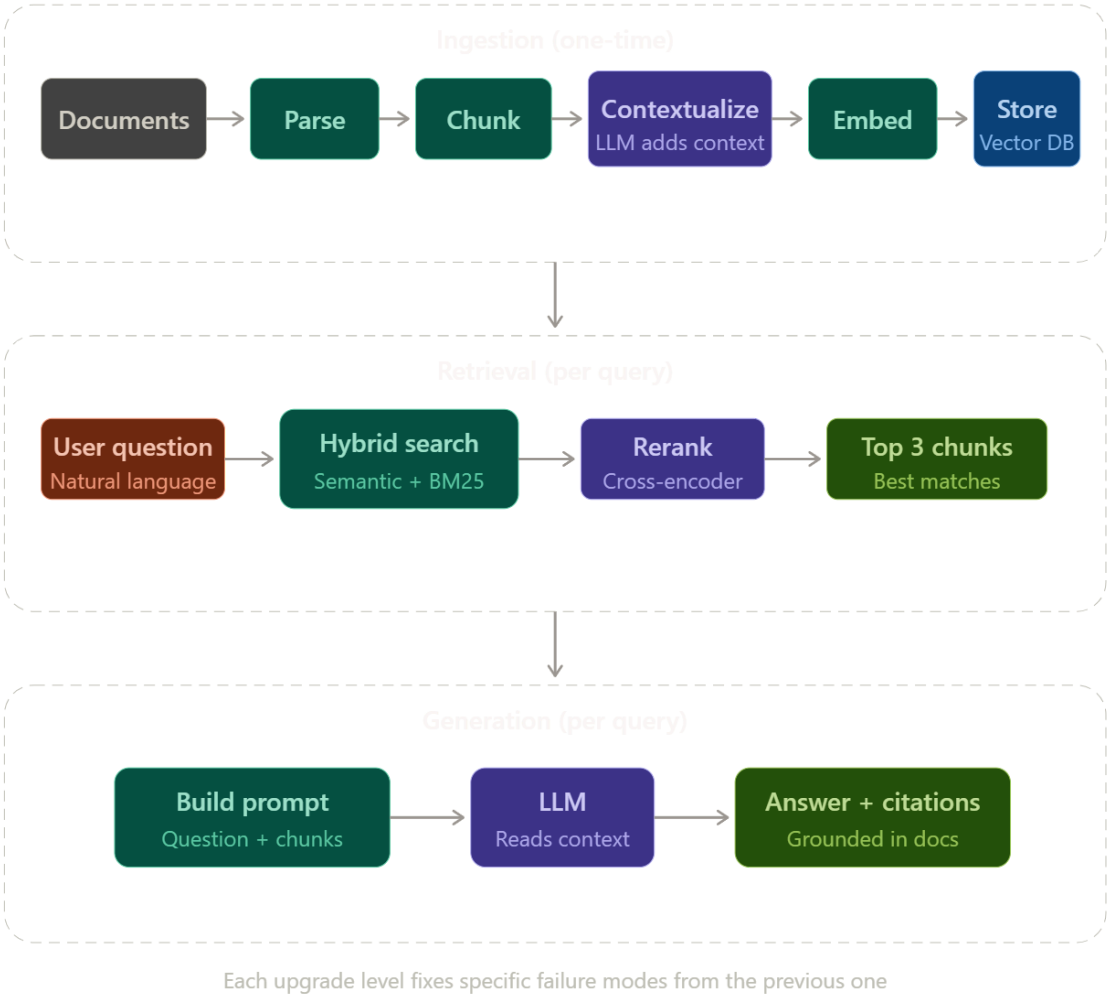
Component	What It Does	Analogy
Parsing	Convert raw documents (PDF, HTML, Word) to clean text	Cleaning ingredients before cooking
Chunking	Split documents into searchable pieces	Cutting a book into index cards

Component	What It Does	Analogy
Embeddings	Convert text to vectors that capture meaning	Plotting ideas on a map where similar ideas are nearby
Vector DB	Store and search embeddings at scale	A library with an instant "find similar" button
Semantic Search	Find chunks with similar meaning	Finding books about the same topic
BM25	Find chunks with exact word matches	Finding books with an exact phrase
Hybrid Search	Combine semantic + BM25 with RRF	Using both the topic index AND the word index
Contextual Retrieval	Prepend LLM-generated context to each chunk before embedding	Writing "From Chapter 3 of..." on each index card
Reranking	Cross-encoder re-scores top results with deeper analysis	Interviewing shortlisted candidates in detail
Query Transformation	Rewrite/expand the question for better search	Asking the librarian to rephrase your question
Prompt Design	Instruct the LLM to answer ONLY from context, cite sources	Telling a student "use ONLY the provided notes for your essay"

Homework from the Lecture

1. **BUILD:** Full RAG pipeline over YOUR documents. Try two chunking strategies (fixed vs recursive) and compare retrieval quality.
2. **READ:** anthropic.com/news/contextual-retrieval — the full Anthropic blog post on contextual retrieval.
3. **THINK:** Where did your pipeline fail? Which specific upgrade (hybrid search? reranking? contextual?) would fix it?

KEY DIAGRAMS:



Here's the full pipeline. Now the upgrade path showing what each level fixes:

Level 6: Query transformation, Level 7: Agentic RAG